

Validation

Validation

- Prüfung der Daten bevor sie in Source geschrieben werden
- Insb. zur Überprüfung von Benutzereingaben
- Zeigt im Fehlerfall ErrorTemplate
- Kann im Fehlerfall eine Fehlerbenachrichtigung zurückgeben
- 3 Möglichkeiten:
 - ValidationRule in eigener Klasse definieren
 - ValidateOnException
 - ValidateOnDataError

Die Validation-Mechanik in WPF ermöglicht die Überprüfung von Benutzereingaben in Steuerelementen, um sicherzustellen, dass die eingegebenen Daten den definierten Regeln und Anforderungen entsprechen. Sie stellt sicher, dass die Daten in einem bestimmten Zustand gültig sind, bevor sie in das zugrunde liegende Datenmodell übernommen werden. Dies erfolgt immer in der Richtung von dem Target zur Source.

Die Überprüfung kann auf verschiedenen Ebenen stattfinden. Auf der Eigenschaftsebene können Validierungsregeln wie erforderliche Felder, Mindest- oder Höchstwerte oder bestimmte Muster definiert werden. Auf der Objektebene können komplexe Validierungslogik oder Abhängigkeiten zwischen mehreren Eigenschaften überprüft werden. WPF bietet auch vordefinierte Validierungsregeln, wie z.B. die Überprüfung auf leere Eingaben oder das Festlegen von Datums- oder Zahlenformaten.

Die Validierungsergebnisse werden in der Benutzeroberfläche angezeigt, typischerweise durch das Einfärben des Steuerelements oder das Anzeigen einer Fehlermeldung (innerhalb sog. ErrorTemplates). WPF stellt auch die Möglichkeit bereit, von den Validierungsmechaniken zurückgegebene Fehlermeldungen in der Oberfläche darzustellen.

Die Validierung in WPF erfolgt normalerweise in Echtzeit, was bedeutet, dass die Überprüfung sofort stattfindet, wenn der Benutzer Daten eingibt oder ändert. Dadurch wird sichergestellt, dass der Benutzer sofort über etwaige Validierungsfehler informiert wird und korrigierende Maßnahmen ergreifen kann.

ValidationRule

- Implementierung der abstrakten Klasse *ValidationRule*
- Methode *Validate* überprüft Wert auf Anforderungen und gibt *ValidationResult* zurück
- *ValidationResult* beinhaltet Ergebnis der Validierung und gegebenenfalls Fehlermeldung
- Einbindung in XAML:

```
<TextBox x:Name="TextBoxIP" MaxLength="15">
    <TextBox.Text>
        <Binding Path="IPAdresse" UpdateSourceTrigger="PropertyChanged">
            <Binding.ValidationRules>
                <local:IPValidationRule/>
            </Binding.ValidationRules>
        </Binding>
    </TextBox.Text>
</TextBox>

<Label Content="{Binding ElementName=TextBoxIP, Path=(Validation.Errors)[0].ErrorContent}"/>
```

Um eine eigene Validierungsregel zu definieren, lässt man eine Klasse von der abstrakten Klasse ‚ValidationRule‘ erben. In die nun zu implementierende Methode ‚Validate‘ wird die entsprechende Validierungslogik eingefügt. Die Rückgabe ist vom Typ ‚ValidationResult‘ und besteht entweder aus der Bestätigung der erfolgreichen Validierung oder aus der Verneinung dieser in Kombination mit einer Fehlermeldung, die z.B. in einem ErrorTemplate angezeigt werden kann. Die AttachedPropertie Validation.Errors beinhaltet sämtliche Validierungsfehler als Array.

In die Property ‚ValidationRules‘ des Binding-Objekts können beliebig viele Validierungsregeln eingesetzt werden, welche dann nacheinander überprüft werden bevor die Bindung vollständig ausgeführt wird, also bevor der Wert in die Source übertragen wird.

Validierungsregeln werden für kleinere, formale Überprüfungen verwendet, wie z.B. Datentypprüfungen, Stringlängen, Musterprüfungen, etc. , sowie für alle weiteren Prüfungen, die vor dem Schreiben in die Source stattfinden sollen. Sie sind auch die einzige Mechanik, die auf PropertyBindings angewendet werden kann.

ValidateOnException

- Exceptionwurf, wenn ein Wert nicht den Anforderungen entspricht (z.B. in einem Setter der Model-Klasse)

```
private string name;  
public string Name  
{  
    set  
    {  
        if (value.All(x => Char.IsLetter(x))) name = value;  
        else throw new Exception("Bitte gib nur Buchstaben ein.");  
    }  
}
```

- Kann beliebige Exception sein
- ErrorMessage ist gleichzeitig die Fehlerbenachrichtigung für den User
- Einbindung in XAML:

```
<TextBox Text="{Binding Name, ValidatesOnExceptions=True}"/>
```

Die Eigenschaft "ValidateOnExceptions" ist eine Eigenschaft des Binding-Objekts, die bestimmt, ob eine Validierungsmechanik für eine Eigenschaft ausgelöst werden soll, wenn eine Exception während der Aktualisierung des Bindungsziels auftritt.

Wenn "ValidateOnExceptions" auf "True" gesetzt ist, wird die Validierungsregel unabhängig von anderen Fehlern oder Warnungen ausgelöst, die während des Aktualisierungsvorgangs auftreten können. Dies bedeutet, dass eine Exception, die während des Setzens des Eigenschaftswerts auftritt, dazu führt, dass die Validierungsregel aktiviert wird und das Ergebnis als ungültig betrachtet wird.

Die Verwendung von "ValidateOnExceptions" ist besonders nützlich, wenn es um die Validierung von Benutzereingaben geht, da sie sicherstellt, dass Ausnahmen, die während der Datenaktualisierung auftreten, als ungültig behandelt werden und entsprechende Fehlermeldungen oder visuelles Feedback angezeigt werden können. Insbesondere bei Anbindung an Third-Party-Klassen kann dies z.T. die einzige Möglichkeit sein, Exceptions abzufangen.

Es ist wichtig zu beachten, dass die Verwendung von "ValidateOnExceptions" voraussetzt, dass die Eigenschaft, auf die das Binding angewendet wird, die Ausnahmen, die während der Aktualisierung auftreten können, tatsächlich wirft. Dies kann zum Beispiel der Fall sein, wenn die Eigenschaft eine Abhängigkeitseigenschaft ist und ihre Set-Methode entsprechend implementiert wurde.

Die ausgegebene Fehlermeldung ist der Inhalt der „Message“-Eigenschaft des Exception-Objekts.

ValidateOnDataErrors

- Insb. für Modelklassen gedacht
- Implementierung des Interfaces *IDataErrorInfo*
- Indexer-Property zur Validierung:

```
public string this[string columnName]
{
    get
    {
        if (columnName == nameof(Alter)
            if (Alter < 0 || Alter > 150)
                return "Bitte gib dein wahres Alter an.";
            return null;
        }
    }
}
```

- Einbindung in XAML:

```
<TextBox Text="{Binding Name, ValidatesOnDataErrors=True}"/>
```

IDataErrorInfo ist ein Schnittstelleninterface, das es ermöglicht, die Validierung von Datenobjekten auf der Ebene der Datenklassen bzw. ViewModels zu implementieren. Durch die Implementierung von *IDataErrorInfo* kann ein Datenobjekt Fehlerinformationen für seine einzelnen Eigenschaften bereitstellen.

Die Schnittstelle *IDataErrorInfo* enthält zwei Eigenschaften: *Error* und *Item[]*. Die *Error*-Eigenschaft wird in WPF nicht explizit verwendet, kann aber z.B. verwendet werden um einen allgemeinen Fehler für das gesamte Datenobjekt zurückzugeben. Die *Item[]*-Eigenschaft, eine Indexer-Eigenschaft, definiert Validierungsregeln für einzelne Eigenschaften und gibt die spezialisierten Fehlermeldungen zurück.

Dazu wird gemeinhin ein switch in deren Getter etabliert, der auf den *columnName* zielt, in welchem sich die Bezeichnung der zu validierende Eigenschaft als String befindet. Die cases beinhalten dann die Validierungslogiken. Gibt diese Eigenschaft einen nicht-leeren String zurück, dann wird das als Validierungsfehler gewertet und der String wird als Fehlerbenachrichtigung an die GUI gesendet. Ein leerer String oder null werden als erfolgreiche Validierung gewertet.

Im Binding-Objekt muss die Eigenschaft ‚*ValidateOnDataErrors*‘ auf true gesetzt werden, damit diese Interface-Mechanik verwendet wird.

Diese Methodik ermöglicht eine komplexere Validierung als die *ValidationRules*, da hier z.B. Bezug auf andere Eigenschaften genommen werden kann. Allerdings erfolgt die Validierung hier erst, nachdem der Wert schon in die Source übertragen wurde.

ErrorTemplates

- ControlTemplates, welcher im Fehlerfall **über** das Original-Template gelegt wird
- Original-Control befindet sich in *AdornedElementPlaceholder* innerhalb des ErrorTemplates

```
<ControlTemplate x:Key="Ctp_Error_01">
    <StackPanel>
        <Border BorderBrush="Red" BorderThickness="2">
            <AdornedElementPlaceholder x:Name="Aep"/>
        </Border>
        <TextBlock Foreground="Red"
            Text="{Binding ElementName=Aep,
                Path=AdornedElement.(Validation.Errors)[0].ErrorContent}"/>
    </StackPanel>
</ControlTemplate>
```

- Zugriff auf Fehlertexte über *Validation.Errors* des Original-Controls

ErrorTemplates sind eine Möglichkeit, benutzerdefinierte visuelle Darstellungen für Validierungsfehler oder Fehlermeldungen anzuzeigen. Wenn bei der Validierung eines Datenobjekts ein Fehler auftritt, wird das ErrorTemplate verwendet, um den visuellen Stil des Elements zu ändern, das den Fehler darstellt.

Ein ErrorTemplate ist im Normalfall ein ControlTemplate welches in die AttachedProperty ‚Validation.ErrorTemplate‘ gesetzt wird. Zentral beinhaltet es einen AdornedElementPlaceholder-Objekt, welches in der Anwendung das Original-Control beinhaltet. Umgeben wird dieses dann mit visuellen Elementen (z.B. roter Rahmen), um auf den Fehler hinzuweisen. Ergänzt werden kann dies um Elemente, die man an die Validation.Errors-Eigenschaft anbindet, um die Fehlernachrichten auszugeben.

Durch die Verwendung von ErrorTemplates können Entwickler die Darstellung von Validierungsfehlern vollständig anpassen, um sie in das Gesamtdesign der Anwendung zu integrieren oder spezifische Benutzervorgaben zu erfüllen. Dies ermöglicht eine verbesserte Benutzererfahrung, da Benutzer sofort erkennen können, welche Daten nicht korrekt eingegeben wurden und wie sie das Problem beheben können.